Build Prompt: `mini-coding-js` — a tiny Node coding-agent harness

> **How to use this document:** give everything below (from "ROLE" onward) to a capable coding model as a single build prompt. It is a reverse-engineered specification of the existing `mini-coding-agent-js` app: a deliberately small, readable, teachable coding agent implemented in plain Node.js with one dependency (`chalk`). The goal is not merely an agent that works, but one whose architecture makes the lesson visible: **one ReAct loop, eight tools, a write sandbox, nine lifecycle hooks, persistent memory, and an offline demo that proves the guards hold.**

CONTENTS

What is in this explainer

ROLE 4

PRIMARY CONSTRAINTS 5

DELIVERABLE FILE LAYOUT 6

ARCHITECTURE AT A GLANCE 7

Workspace contract 8

Constants and errors 9

Sandbox contract 10

Shell write-target scanning 11

Exact tool set 12

Tool contracts 13

Argument validation and dispatch 16

chat 16

sanitizeAssistant 16

Message assembly 20

The seven beats, once per step 21

Entry guard 22

Nine hook call sites 23

Contract 24

The nine events 25

Hooks registry 26

Tracing and catalogue 27

Store format 28

Defaults 29

Memory API 30

.env discovery 32

.env loading 33

Fingerprinting 34

Model normalisation 35

loadConfig(overrides = {}, { envFile, startDir }) 36

Palette/API 37
Result rendering 38
explainExit(code) 39
Demo script: 14 tool steps, then a final answer 41
Scorecard 41
makeConfirm() 43
askTask({ input, output }) 44
Argument parsing 45
Help/usage 46
Side doors 47
Banner 48
Wiring 49
Sessions 50
Exit codes 51
workspace.test.js 52
sandbox.test.js 53
tools.test.js 54
shell-sandbox.test.js 55
agent.test.js 56
hooks.test.js 57
memory.test.js 58
prompt.test.js 59
llm.test.js 60
config.test.js 61
session.test.js 62
ui.test.js 63
input.test.js 64

Page numbers refer to the folio at the foot of each page.

ROLE

You are a senior Node.js engineer building a from-scratch implementation of a minimal coding-agent harness. Build the complete application described here. Do not substitute a framework, an agent SDK, a database, a build system, or a frontend. Prefer the Node standard library. The only runtime dependency is `chalk` for terminal colour.

The app must be small enough for a student to read in one sitting, but complete enough to demonstrate the real architecture of a coding agent: a model/tool loop, a system prompt, an injected model client, a tool dispatcher, path and command guardrails, lifecycle hooks, session history, and a human-readable long-term memory.

PRIMARY CONSTRAINTS

1. **Language/runtime:** modern ES modules (`"type": "module"`), Node.js **>= 20**, JavaScript only. No TypeScript, no transpiler, no bundler.
2. **Dependencies:** exactly one runtime dependency: `chalk` (v5+). Everything else uses `node:` built-ins.
3. **Provider:** OpenRouter's OpenAI-compatible `/chat/completions` endpoint, called with plain `fetch`. No provider SDK.
4. **Separation of concerns:** the loop must be a library. It must not parse flags, print a banner, choose colours, or call `process.exit`.
5. **Dependency injection:** the loop receives `workspace`, `sandbox`, `commandPolicy`, `maxSteps`, `chatFn`, `confirm`, `hooks`, `memory`, `history`, `log`, and `heartbeatMs`. Tests must be able to run the real loop with a fake model and no network.
6. **No hidden state:** memory is a markdown file, read from disk on every operation. History is a plain message array. Configuration is resolved once and passed as an object.
7. **Everything the model sees is text:** each tool returns a string observation. Tool errors are returned as strings, never thrown out of the loop.
8. **Offline-first testability:** the full test suite runs with `node --test`, no API key, no network.
9. **Readable design:** comments should explain *why* a module exists and what contract it owns, not narrate syntax.
10. **No fake safety claims:** the sandbox must document exactly what it does and does not protect against.

DELIVERABLE FILE LAYOUT

Create this structure:

```
mini-coding-agent-js/
├── package.json
├── .gitignore
├── README.md
├── demo_prompt.md
├── src/
│   ├── agent.js          # the ReAct loop (library, one exported runTask)
│   ├── cli.js            # the program: flags, banner, sessions, exit codes
│   ├── config.js         # .env discovery/loading, model normalisation
│   ├── demo.js           # scripted fake model + sandbox scorecard
│   ├── heartbeat.js      # "still waiting" liveness timer
│   ├── hooks.js          # lifecycle registry, catalogue, tracer
│   ├── llm.js            # OpenRouter fetch call + assistant sanitisation
│   ├── memory.js         # markdown-backed long-term memory
│   ├── memory.md         # the initial memory store
│   ├── prompt.js         # system prompt + memory digest injection
│   ├── sandbox.js        # write fence + shell write-target detection
│   ├── tools.js          # 8 tool schemas + handlers + validation
│   ├── ui.js             # all terminal styling
│   └── workspace.js      # path policy (read vs write)
├── test/                 # offline node:test suites
└── workspace/
    └── .gitkeep          # the agent's writable root
```

`package.json` must contain:

```
{
  "name": "mini-coding-js",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "description": "A tiny Node coding-agent harness: ReAct loop + 8 tools, one dependency for colour.",
  "scripts": {
    "start": "node src/cli.js",
    "demo": "node src/cli.js --demo",
    "hooks": "node src/cli.js --hooks",
    "memory": "node src/cli.js --memory",
    "trace": "node src/cli.js --demo",
    "test": "node --test"
  },
  "engines": { "node": ">=20" },
  "dependencies": { "chalk": "^5.6.2" }
}
```

`.gitignore` must ignore `node_modules/`, `.env`, everything under `workspace/*`, but keep `!workspace/.gitkeep`.

ARCHITECTURE AT A GLANCE

```
CLI (cli.js)
  ├─ loadConfig()              config.js
  ├─ Workspace + Sandbox       workspace.js / sandbox.js
  ├─ Memory                    memory.js
  ├─ chatFn (real | demo)      llm.js / demo.js
  ├─ Hooks (trace by default)  hooks.js
  └─ runTask() per task        agent.js
       └─ think → model call → tool calls? ──no──→ answer, stop
                    ↑                 |
                    └──── yes: run each tool, append result
```

The loop must be generic and testable. The CLI owns all process concerns. `agent.js` must remain unaware of flags, `.env`, colours, and exit codes.

---

1. `workspace.js` — the path policy

Export `PathError` and `Workspace`.

`Workspace` contract

```
new Workspace(root) // resolves root, mkdir -p, stores `root` as a realpath
workspace.resolve(requested)     // returns { abs, rel }; mutations only
workspace.resolveRead(requested) // returns { abs, rel }; reads anywhere
```

- Relative paths resolve against the workspace root.
- `rel` is `/`-separated and is `'.'` for the root.
- `resolve()` is the strict write policy. It must reject:
- empty/non-string paths;
- drive-absolute and drive-relative paths (`C:\...`, `C:foo`);
- UNC paths (`\\server\share`, `//server/share`);
- any absolute path;
- paths beginning with `~`, `$`, `%` that expand to a home/environment path;
- a first segment that looks like a home/system folder: `users`, `windows`, `program files`, `program files (x86)`, `programdata`, `desktop`, `documents`, `downloads`, `appdata`, `onedrive`, `library`, `system32`;
- any path that escapes the root after realpath resolution.
- Expand `~`, `%VAR%`, and `${VAR}` / `$VAR` before containment checks.
- Symlink/junction safety: resolve the deepest existing ancestor with `fs.realpathSync`, then append the not-yet-created remainder. A symlink inside the workspace must not smuggle a write outside.
- Containment test: `rel === ''` or `rel` does not start with `..` and is not absolute.
- `resolveRead()` deliberately does **not** fence reads. Absolute paths and paths outside the workspace are allowed. This is intentional: an agent must be able to read docs/logs/libraries; only mutation is dangerous. Relative reads still start at the workspace root.

The read/write asymmetry is a core lesson: **reads are open; writes are fenced.**

---

2. `sandbox.js` — the mutation gate

Export `SANDBOX_DIR`, `ACTIONS`, `ForbiddenError`, `Sandbox`, `sandboxFor`, `tokenizeCommand`, `shellWriteTargets`.

Constants and errors

```
export const SANDBOX_DIR = path.resolve(import.meta.dirname, '..', 'workspace');
export const ACTIONS = Object.freeze({ SAVE: 'save', EDIT: 'edit', DELETE: 'delete' });
```

`ForbiddenError` carries `action`, `requested`, `reason`, and has `name = 'ForbiddenError'`, `code = 'FORBIDDEN'`. Its message is exactly:

```
SAVE ../escape.txt - path escapes the workspace: ../escape.txt
```

`Sandbox` contract

```
new Sandbox({ workspace, root, log }) // reuses workspace or builds one from root; one per run
sandbox.refusals                      // count of refusals
sandbox.check(action, requested)      // { abs, rel } or throws ForbiddenError
sandbox.write(target, content)        // mkdir -p, write, return { existed, bytes }
sandbox.save(requested, content)
sandbox.remove(requested)             // file or empty dir; only inside root
sandbox.checkCommand(command)         // inspect visible shell write targets
sandbox.summary()                     // "save, edit, delete - only inside <root>"
Sandbox.allowsAction(action)
sandboxFor(ctx)
```

- `check()` delegates path policy to `workspace.resolve()` and converts `PathError` into a refusal.
- Deleting the sandbox root itself must be refused.
- A refusal must: 1. increment `refusals`; 2. print exactly one loud line through `ui.forbidden(...)`; 3. throw `ForbiddenError`.
- This is the **only** code path that prints `FORBIDDEN`. That gives the question "was this allowed?" one answer and one place to look.
- `remove()` on directories uses `rmdirSync` (empty directories only, on purpose). Missing targets return `{ missing: true }`.
- `checkCommand()` is best effort. It tokenises the command line and finds visible write targets. If any target resolves outside the workspace, print the `FORBIDDEN` line(s), then throw one `ForbiddenError('command', joinedTargets, 'the shell would write outside the sandbox')`. Refuse *before* asking for approval.

Shell write-target scanning

`tokenizeCommand(text)` must split on whitespace while honouring single and double quotes, and emit operators as tokens: `>`, `>>`, `<`, `|`, `||`, `&`, `&&`, `;`. Quotes are stripped, so quoted paths with spaces remain one token.

`shellWriteTargets(command)` scans each command segment between separators:

- Redirections: `>` and `>>` write to the following token. Handle `2> file` where the stream number is a separate token.
- `<` writes nothing; skip its operand.
- `DEST_LAST` commands: `copy`, `cp`, `move`, `mv`, `xcopy`, `ren`, `rename`, `ln`, `install`, `scp`, `rsync`. Their last positional argument is the write target; require at least two positionals.
- `robocopy`: positional argument 2 is the destination.
- `ALL_POSITIONAL` commands: `del`, `erase`, `rd`, `rmdir`, `rm`, `md`, `mkdir`, `touch`, `tee`, `unlink`, `truncate`, `shred`. Every positional argument is a write/delete target.
- Downloaders: `curl` output flags `-o`, `--output`, `-O`, `--remote-name`; `wget` `-O`, `--output-document`. For `curl -O`/`--remote-name`, the output is the URL filename (last positional).
- PowerShell: `out-file`, `set-content`, `add-content`, `new-item`, `ni`, `remove-item`, `copy-item`, `move-item`, `rename-item`; target flags `-destination`, `-dest`, `-path`, `-filepath`, `-literalpath`. Also inspect nested cmdlets inside `powershell`/`pwsh`/`ps`.
- Strip `.exe`, `.cmd`, `.bat`, `.com`, `.ps1` from command names and compare case-insensitively.
- A switch is `--flag`, `-f`, or a short Windows `/x`; a POSIX path like `/home` is not a switch.

Document the limitation honestly: this is not a shell sandbox. `node -e "fs.copyFileSync(...)"` or a downloaded script can still write anywhere. A container is the real answer. Do not claim otherwise.

---

3. `tools.js` — eight tools and a dispatcher

Export `TOOL_SCHEMAS`, `TOOL_NAMES`, `SANDBOX_LIMITS`, `safeEnv`, `runTool`.

Exact tool set

The model may call exactly these eight tools, in this order in the schema array and in `TOOL_NAMES`:

1. `list_files`
2. `read_file`
3. `create_file`
4. `write_file`
5. `edit_file`
6. `run_command`
7. `remember`
8. `recall`

Each schema is OpenAI function-calling shaped:

```
{ type: 'function', function: { name, description, parameters: {...} } }
```

All parameter objects must use `additionalProperties: false`.

Tool contracts

`list_files`

Parameters: `path` (string, optional, default `'.'`), `depth` (integer, optional, default 3, max 6).

- Resolve with `workspace.resolveRead()`.
- Missing folder → `ERROR: no such folder: <rel>`; non-folder → `ERROR: not a folder: <rel>`.
- Walk recursively, sorted by name, skipping `.git`, `node_modules`, `.venv`, `__pycache__`.
- Directories print `<rel>/`; files print `<rel> (<size>b)`.
- Stop at `depth` levels and at 200 lines.
- End with `\n\n<dirs> folder(s), <files> file(s) under <start.rel>`.
- Empty result is `(empty)` plus the counts.

`read_file`

Parameters: `path` (required), `offset` (integer, default 1), `limit` (integer, default 200, max 2000).

- Resolve with `resolveRead()`.
- Missing → `ERROR: no such file: <rel>`; directory → `ERROR: <rel> is a folder - use list_files`.
- Return `"<rel> (<total lines>)"`, then numbered lines with line numbers right-aligned to width 4 followed by `|`.
- Append `\n... [<n> more lines]` when truncated.
- `offset` clamps to >= 1; `limit` clamps to 1..2000.

`create_file`

Parameters: `path`, `content` (both required).

- Use `sandbox.check(ACTIONS.SAVE, path)`.
- If the file exists, return `ERROR: <rel> already exists - use edit_file to change it or write_file to overwrite` and do not touch it.
- Otherwise write through the sandbox; return `created <rel> (<bytes> bytes)`.

`write_file`

Parameters: `path`, `content` (both required).

- Sandbox save check, create parent folders, overwrite.
- Return `created <rel> (<bytes> bytes)` or `overwrote <rel> (<bytes> bytes)`.

`edit_file`

Parameters: `path`, `old_string`, `new_string`, `replace_all` (boolean, default false).

- Sandbox `EDIT` check. Missing file → `ERROR: no such file: <rel>`.
- Empty `old_string` → `ERROR: old_string must not be empty`.

- Count exact occurrences.
- 0 occurrences → `ERROR: old_string not found in <rel>. Read the file and copy the snippet exactly, including whitespace.`
- > 1 and no `replace_all` → `ERROR: old_string appears N times in <rel>. Add surrounding context to make it unique, or pass replace_all: true.`
- Replacement must be literal: use `source.replace(oldStr, () => newStr)` or `split().join()` so `$1`, `$&` etc. are not interpreted.
- Return `edited <rel>: <n> replacement(s), first at line <line>`.

`run_command`

Parameter: `command` (required string).

- Trim; empty → `ERROR: command is required`.
- Apply a hard deny-list **before** policy and approval. Return `DENIED: blocked by policy (<why>). Command: <command>`.
- Call `sandbox.checkCommand(command)` before asking a human, so no one approves an impossible write.
- Policy:
- `deny` → `DENIED: command execution is disabled (CMD_POLICY=deny).`
- `allow` → proceed.
- `ask` → `await ctx.confirm(command)`; anything other than `'allow'` → `DENIED: the user did not approve this command. Command: <command>`.
- Execute with `node:child_process.exec` in `workspace.root`, `timeout: 30_000`, `maxBuffer: 1 MB`, `windowsHide: true`, `encoding: 'utf8'`, and **only the allow-listed environment**.
- Result format:

```
exit code <n>
stdout:
<clipped stdout>

stderr:
<clipped stderr>
```

Add only non-empty streams; if both are empty, add `(no output)`. - Clip each stream to 6,000 characters and append `... [<n> chars truncated]`. - Timed-out commands must append `[tool] command timed out after 30s` to stderr and return a non-zero code.

Hard deny-list

At least these patterns, with these explanations:

PATTERN	WHY
recursive delete of a root path	`recursive delete of a root path`
`del /...s` or `/q`	`recursive/quiet Windows delete`

PATTERN	WHY
`format`, `diskpart`	`disk formatting`
`shutdown`, `reboot`	`machine power control`
`git push --force` / `-f`	`force push`
`curl` / `wget` piped into `sh` / `bash`	`piping the internet into a shell`
`:(){ ...};:`	`fork bomb`
`> /dev/sd...`	`raw disk write`

The deny-list applies regardless of `CMD_POLICY`, including `allow`.

Environment allow-list

A child process must not inherit the harness environment; otherwise `node -e "console.log(process.env.OPENROUTER_API_KEY)"` leaks the key. `safeEnv(source)` keeps only:

```
PATH, PATHEXT, HOME, USERPROFILE, TMPDIR, TEMP, TMP,
SYSTEMROOT, COMSPEC, SHELL, LANG, LC_ALL, TERM
```

Export `safeEnv` and test it directly.

`remember`

Parameters: `text` (required), `tag` (optional; one of `fact`, `pref`, `fix`, `gotcha`).

- If `ctx.memory` is null: `ERROR: no memory store in this run (started with --no-memory)`.
- Delegate to `memory.remember()`.
- Empty → `ERROR: <reason>`.
- Duplicate → `already remembered in <label> (<total> total), date refreshed: <formatted entry>`.
- New → `remembered in <label> (<total> total[, dropped the N oldest]): <formatted entry>`.

`recall`

Parameters: `query` (string, optional), `limit` (integer, default 20).

- No memory → same error as `remember`.
- No hits → `no memories in <label> match "<query>" (<size> stored)`.
- One hit → `<n> of <size> memories in <label>: <formatted entry>` on one line.
- Multiple hits → header line, then one formatted entry per line.

Argument validation and dispatch

- Build a parameter metadata map from the schemas: name → `{ argName: type }`.
- `validate(name, raw)`:
- raw may be a JSON string or an object;
- invalid JSON → `ERROR: could not parse arguments for <name>: <message>`;
- non-object / array / null → `ERROR: arguments for <name> must be a JSON object`;
- for each provided value, check JavaScript type; integers must satisfy `Number.isInteger`; return `ERROR: "<key>" must be a <type>, got <actual>`.
- Missing values are allowed (handler defaults apply).
- `runTool(name, rawArgs, ctx)`:
- unknown tool → `ERROR: unknown tool "<name>". Available: <list>`;
- validation error returned as text;
- handler exceptions become text:
  - `ForbiddenError` → `FORBIDDEN: <message>`;
  - `PathError` → `ERROR: <message>`;
  - other → `ERROR: <code|failed>: <message>`.
- Every handler is `(workspace, args, ctx)` and returns a string or a promise of one.

---

4. `llm.js` — the model call

Export `chat` and `sanitizeAssistant`.

`chat`

```
chat({ apiKey, model, messages, tools, signal, timeoutMs = 90_000 })
```

- Endpoint: `https://openrouter.ai/api/v1/chat/completions`.
- Missing key → throw `OPENROUTER_API_KEY is not set (put it in .env or the environment)`.
- Combine an optional caller `signal` with `AbortSignal.timeout(timeoutMs)` using `AbortSignal.any`.
- POST JSON with headers:
- `Authorization: Bearer <key>`
- `Content-Type: application/json`
- `X-Title: min-coding-js`
- Body: `{ model, messages, tools, max_tokens: 2048 }`.
- On fetch timeout, throw `request timed out after <seconds>s - OpenRouter did not answer`.
- Non-2xx: throw `OpenRouter <status><hint>: <first 400 chars of body>`, where hint is:
- 401 → `(bad or revoked key)`
- 402 → `(out of credits)`
- Non-JSON body → `OpenRouter returned non-JSON: <first 200 chars>`.
- JSON containing `.error` → `OpenRouter error: <serialised, first 400 chars>`.
- Return the parsed JSON.

`sanitizeAssistant`

Rebuild the assistant message with only `role: 'assistant'`, `content`, and — only when non-empty — `tool_calls`. Delete `content` when it is null and tool calls exist. Drop `reasoning` / `reasoning_content` so old traces are not re-sent on every later step.

---

5. `prompt.js` — the system prompt

Export `systemRules`, `buildSystemPrompt`.

`systemRules(tools = TOOL_NAMES)` must produce prose close to this (exact wording is part of the behaviour):

```
You are a small coding agent working inside a sandboxed folder.

You have tools: <comma-separated tool names>. Reads may use any path (relative paths start at your workspace root;
absolute paths are allowed). Writing is fenced: save, edit and delete only work inside the workspace, and anything else
is refused with FORBIDDEN - do not retry it, work inside the workspace instead.

How to work:
1. Look before you write: list_files / read_file before editing.
2. Prefer create_file for new files, edit_file for small changes, write_file only when replacing a whole file.
3. Every edit must use an old_string copied exactly from read_file output.
4. Run what you wrote with run_command and check the exit code. If it fails, read the error, fix it, run again.
5. Do not invent tool results. Stop calling tools when the task is done and reply with a short summary of what you
changed.
6. If you learn something that will still be true next week - a command this repo needs, a preference the user stated,
a trap you fell into - save it with remember. Never save task progress: "step 3 of 5" is not a memory, and it is
worthless to the next run.

Be concise. One tool call per step where possible.
```

`buildSystemPrompt({ memory, log })`:

- Always return the rules if no memory instance or no digest.
- Otherwise append the memory digest after a blank line.
- Count injected lines that start with `- [` and, when `log` is supplied, emit exactly one `ui.memory(...)` line: `<label> - N memory/memories added to the system prompt`.

Keeping this prose out of `agent.js` is deliberate: the loop should read as control flow, not three screens of prompt engineering.

---

6. `agent.js` — the loop

Export exactly one main function:

```
runTask({
  task,
  workspace,
  sandbox = null,
  commandPolicy,
  maxSteps,
  chatFn,
  confirm,
  hooks = null,
  memory = null,
  history = [],
  log = console.log,
  heartbeatMs = 5000,
})
```

Return shape:

```
{ ok: true,  answer, messages, steps }
{ ok: false, reason: 'blocked_by_hook' | 'request_failed' | 'max_steps', messages }
```

Message assembly

At the start of every task:

```
const messages = [
  { role: 'system', content: buildSystemPrompt({ memory, log }) },
  ...history.filter((m) => m.role !== 'system'),
  { role: 'user', content: task },
];
const taskIndex = messages.length - 1;
```

- The system prompt is always rebuilt; it is never carried in history. A memory written by the previous task therefore appears before the next task.
- `history` is the previous task's conversation, excluding system messages.
- Build a tool context:

```
const ctx = {
  workspace,
  sandbox: sandbox ?? new Sandbox({ workspace, log }),
  commandPolicy,
  memory,
  confirm: confirm ?? (async () => 'deny'),
};
```

- Record a start timestamp and a duration helper.
- Use `hooks ?? new Hooks()` so every call site can call `hook.run(...)` unconditionally.

The seven beats, once per step

For `step = 1..maxSteps`:

1. Print a blank line and `ui.step(step, maxSteps)`.
2. Fire `PreModel` with `{ step, maxSteps, messages, tools: TOOL_SCHEMAS }`.
3. Start `heartbeat(log, heartbeatMs)` and call `chatFn({ messages, tools: TOOL_SCHEMAS })`. - On throw: stop the heartbeat, print `ui.error('request failed: ...')`, fire `SessionEnd` with `ok: false, reason: 'request_failed', steps: step, durationMs, messages`, return `{ ok: false, reason: 'request_failed', messages }`. - On success: stop the heartbeat.
4. Extract `choice = reply?.choices?.[0] ?? {}` and `message = choice.message ?? {}`. Fire `PostModel` with `{ step, message, finishReason: choice.finish_reason }`.
5. **Truncation guard:** if `finish_reason === 'length'` and there are no tool calls: - append `{ role: 'assistant', content: message.content ?? '' }`; - append `{ role: 'user', content: 'You were cut off by the length limit. Continue with smaller tool calls.' }`; - print `ui.warn('reply was truncated - asking the model to continue smaller')`; - `continue` to the next step. Never dispatch a truncated tool call.
6. Append `sanitizeAssistant(message)` to messages. If `message.content` is non-empty, print `ui.think(...)`.
7. Get `calls = Array.isArray(message.tool_calls) ? message.tool_calls : []`. - **No calls:** print a blank line, `ui.answerRule()`, and `ui.answer((message.content ?? '(no answer)').trim())`. Fire `Stop` with `{ answer, steps: step, messages }`, then `SessionEnd` with `{ ok: true, reason: 'answer', steps: step, durationMs, messages }`. Return `{ ok: true, answer: message.content ?? '', messages, steps: step }`. - **Calls:** dispatch each call in order:
   1. `name = call.function?.name ?? '(unnamed)'`; `rawArgs = call.function?.arguments ?? '{}'`.
   2. Print `ui.tool(name, rawArgs)`.
   3. Fire `PreToolUse` with `{ name, args: rawArgs, step }`.
      - If blocked, do not run the tool. The observation is exactly: `DENIED: a PreToolUse hook blocked this call (<reason>). Try another approach.`
      - Otherwise `await runTool(name, rawArgs, ctx)`.
   4. Print `ui.result(result)`.
   5. Fire `PostToolUse` with `{ name, result, step, ok: !/^(ERROR|DENIED)\b/.test(result) }`.
      - If `post.replace !== null`, print a dim annotation and use the replacement as the observation.
   6. Append exactly one `{ role: 'tool', tool_call_id: call.id, content: String(post.replace ?? result) }`. Every `tool_call_id` must get a matching reply, or the next request is invalid.

After the loop exhausts `maxSteps`:

- Print `ui.error('stopped: hit the <maxSteps>-step limit')`.
- Fire `StepLimit` with `{ steps: maxSteps, messages }`.
- Fire `SessionEnd` with `{ ok: false, reason: 'max_steps', steps: maxSteps, durationMs, messages }`.
- Return `{ ok: false, reason: 'max_steps', messages }`.

Entry guard

If `agent.js` is executed directly, print:

```
agent.js is the loop, not the program.
Run the CLI instead:  node src/cli.js --help
```

Do not import `cli.js` from `agent.js`.

Nine hook call sites

`agent.js` must contain exactly these `hook.run` moments, in this order:

1. `SessionStart` — before the first model call.
2. `UserPromptSubmit` — before the loop.
3. `PreModel`
4. `PostModel`
5. `PreToolUse`
6. `PostToolUse`
7. `Stop` — happy ending.
8. `StepLimit` — budget exhausted.
9. `SessionEnd` — every exit path, including hook blocks and request failures.

Blocked `SessionStart` / `UserPromptSubmit` paths must:

- print `ui.error('stopped before the first request: ...')` / `ui.error('stopped: ...')`;
- fire `SessionEnd` with `ok: false, reason: 'blocked', steps: 0, durationMs, messages`;
- return `{ ok: false, reason: 'blocked_by_hook', messages }`.

For a rewritten `UserPromptSubmit`, print `ui.dim(' UserPromptSubmit hook rewrote the task: <task>')` and replace `messages[taskIndex]`.

On `PostModel`, a `PreModel` or `PostModel` hook cannot change the run, but may observe.

---

7. `hooks.js` — lifecycle extension points

Export:

```
HOOK_EVENTS, HOOK_NAMES, hookMeta, Hooks, hookDetail,
traceHooks, printHookCatalogue, HOOK_SUMMARY
```

Contract

A hook is a function attached to a named moment. It receives `(payload, { event })`. Its return value:

- `undefined` / nothing → observe only.
- `{ note: 'text' }` → print `ui.hookNote(name, text)` and collect the note.
- `{ task: 'new text' }` → only meaningful for `UserPromptSubmit`; rewrite the task.
- `{ replace: 'text' }` → only meaningful for `PostToolUse`; replace the observation.
- `{ block: true, reason }` → block the call/run. First block wins; later handlers are skipped.
- A thrown hook is reported (`hook <name> (<id>) threw: <message> - ignored`) and never fatal.

Hooks do not call the model and do not run tools. They are pure payload functions.

The nine events

Each event metadata entry must contain: `name`, `icon`, `color` (a chalk mode name), `when`, `gets`, `may`, `why`. Use these exact names, icons, colours, and meanings:

#	NAME	ICON	COLOUR	WHEN	GETS	MAY	WHY
1	`SessionStart`	🚀	cyan	once, before the first model call	task, workspace, config, log	refuse to start, or post a note	run exists but nothing sent; load rules, open transcript, check budget
2	`UserPromptSubmit`	✍️	blue	once, as the task enters the loop	task	rewrite the task or block	last moment before the model sees the user's words
3	`PreModel`	🧠	magenta	before every model call	step, maxSteps, messages, tools	observe	whole conversation about to go over the wire; context management starts here
4	`PostModel`	💬	cyan	after every reply, before dispatch	step, message, finishReason	observe	text came back; useful for token counts, transcripts, loop detection
5	`PreToolUse`	🛠️	yellowBright	before every tool call	name, args, step	**block**	the one hook that changes what the agent may do; permission layer
6	`PostToolUse`	📦	green	after every tool call	name, result, ok, step	**replace observation**	redact a secret, cap output, annotate failure
7	`Stop`	🏁	greenBright	model answered, loop ending	answer, steps, messages	observe	happy ending; summary, cost report, notification
8	`StepLimit`	⏳	yellow	maxSteps reached without an answer	steps, messages	observe	failures need a lifecycle too; escalate/dump transcript
9	`SessionEnd`	🔒	red	always, finished or not	ok, reason, steps, durationMs, messages	observe	finally block: cleanup, timing, audit logging

`DECIDING` is `SessionStart`, `UserPromptSubmit`, `PreToolUse`, `PostToolUse`.

`Hooks` registry

```
const hooks = new Hooks({ log, catalog = false });
hooks.on(name, fn, { id, once = false }) // returns id; unknown name throws
hooks.off(name, id)                      // boolean
hooks.count(name)
hooks.has(name)
hooks.attached()                         // event names with handlers, in HOOK_NAMES order
await hooks.run(name, payload)
```

- Uniqueness/id: default id is `"<event>#<n>"`.
- `on` throws on unknown event or non-function.
- `run` returns:

```
{
  event, ran, blocked, reason,
  task, replace, notes: [], errors: []
}
```

- Run handlers in registration order. Remove `once` handlers before calling. Await async handlers.
- The first `block` sets `blocked` and `reason` and breaks.
- Do not use an EventEmitter: a hook that can change the run must be asked, not merely notified.

Tracing and catalogue

`hookDetail(event, payload)` provides one useful line per event:

- SessionStart/UserPromptSubmit → task, 72 chars.
- PreModel → `step X/Y - N messages`.
- PostModel → `step X - <finishReason|stop>`.
- PreToolUse → `<name> <args first 60>`.
- PostToolUse → `<name> - <first line of result, 60>`.
- Stop → `<steps> step(s) - <first line of answer, 56>`.
- StepLimit → `stopped at N step(s)`.
- SessionEnd → `ok|failed (<reason>) in <durationMs>ms`.

`traceHooks({ log, detail = hookDetail, explain = true })` returns a `Hooks` with one tracer per event, with ids `trace:<Event>`. Each firing prints exactly one line:

```
  <icon>  [HOOK:<Name>] <when> - <detail>
```

Use `ui.hookTrace` so the tag has the event's colour. This is on by default in the CLI.

`printHookCatalogue({ log, hooks })` prints:

- a title: `lifecycle hooks - 9 moments in one run`;
- an explanation that hooks may observe, rewrite, or block;
- for each event: icon, padded name, when, `receives`, `may`, wrapped `why`, and `← attached` if live;
- the firing order:

```
SessionStart
UserPromptSubmit
[ PreModel → PostModel → PreToolUse → PostToolUse ]
Stop | StepLimit
SessionEnd
```

- a closing sentence: 9 hooks, 4 can change the run, the rest only watch.

`HOOK_SUMMARY = { total: 9, deciding: 4, names: HOOK_NAMES }`.

---

8. `memory.js` — long-term memory

Export `MEMORY_FILE`, `MEMORY_TAGS`, `parseMemory`, `formatEntry`, `Memory`.

Store format

Memory is a plain markdown file, default `src/memory.md`, outside the sandbox. The file **is** the store: no database, no index, no cache. Every method reads from disk, so hand edits are picked up immediately and cannot drift.

One bullet per memory:

```
- [fact] The sandbox demo script lives at demo/hello.js and is safe to regenerate with create_file. _(2026-10-28)_
```

- Tags: `fact`, `pref`, `fix`, `gotcha`; unknown tags become `fact`.
- The date is local `YYYY-MM-DD`.
- Parsing must accept `-` or `*`, optional date, and ignore all prose/headings/comments.
- Normalise whitespace so duplicate text with different spacing is the same memory.

The initial `src/memory.md` must contain the header:

```
# memory

Long-term notes this agent keeps between runs. One bullet per memory:
`- [tag] text _(YYYY-MM-DD)_` where tag is fact | pref | fix | gotcha.

This file IS the store - edit it by hand, delete a wrong line, or commit it.
The agent reads a bounded slice at the start of a run and can search the rest
with the `recall` tool.
```

Seed it with a few plausible example memories. Useful initial entries include:

```
- [pref] The user's name is Craig. _(2026-10-28)_
- [pref] The user lives in Brighton, UK. _(2026-10-28)_
- [gotcha] The run_command shell in this sandbox is Windows cmd.exe, so Unix tools like `cat`, `ls` may fail; use
`type`, `dir` etc. instead. _(2026-10-28)_
- [pref] The user is a Python developer. _(2026-10-28)_
- [pref] The user has two dogs. _(2026-10-28)_
```

Defaults

```
{
  maxEntries: 60,
  maxEntryChars: 300,
  rememberChars: 2000,
  promptChars: 1200,
  promptEntries: 12,
}
```

`Memory` API

```
new Memory({ root = process.cwd(), file = MEMORY_FILE, maxEntries, maxEntryChars,
             promptChars, promptEntries, now = () => new Date() })
memory.label      // path relative to root when possible, else absolute
memory.exists
memory.entries()  // always parse the current file
memory.size
memory.save(entries)
memory.remember(text, { tag })
memory.forget(match)
memory.search(query, { limit = 20 })
memory.digest({ entries, chars })
memory.summary()
```

`remember`

- Normalise and truncate the raw note to `rememberChars`.
- Empty → `{ ok: false, reason: 'nothing to remember - the note was empty', total }`.
- Truncate to `maxEntryChars`.
- Duplicate detection is case-insensitive on normalised text. On duplicate: refresh date and tag, save, return `{ ok: true, duplicate: true, entry, total }`. Do **not** append a second bullet.
- New entry: append, drop the oldest if over `maxEntries`, save, return `{ ok: true, entry, dropped, total }`.

`forget`

Delete every entry whose text contains the match (case-insensitive). Return `{ removed, total }`.

`search`

The lazy path behind `recall`. Split the query into words; every word must match the entry text or equal its tag. Empty query means all entries. Return newest first, then apply `limit` (minimum 1, default 20).

`digest`

The eager path injected into the system prompt.

- No entries or `chars <= 0` → `''` (no header, no noise).
- Take the newest `promptEntries`, newest first.
- Add formatted lines while they fit `promptChars`.
- If any are hidden, append:

```
(<n> older memory/memories not shown - search <label> with the recall tool)
```

- Return:

```
## What you remember from earlier runs
These notes live in <label> and were learned before this task started.
Treat them as strong hints, not facts to trust blindly: the code may have moved on since.
- [tag] text _(date)_
...
```

The digest is bounded twice (count and characters), so a growing file cannot quietly eat the context window.

`summary`

One line: `<label> (<n> memory/memories[, newest YYYY-MM-DD])`.

Memory must be harness-owned. The agent reaches it only through `remember` / `recall`, never through `read_file` / `write_file` (those are fenced to the workspace), so it cannot be clobbered mid-run.

---

9. `config.js` — environment resolution

Export `findEnvFile`, `loadDotEnv`, `fingerprint`, `normalizeModel`, `loadConfig`.

`PROJECT_ROOT` is `path.resolve(import.meta.dirname, '..')`.

`.env` discovery

`findEnvFile(startDir = PROJECT_ROOT)` checks `startDir/.env`; if absent, walks up parent directories to the filesystem root. Nearest file wins.

`.env` loading

`loadDotEnv(file)` supports:

- `KEY=VALUE`;
- optional `export` prefix;
- `#` comment lines;
- quoted values (strip surrounding single/double quotes);
- blank values are ignored.

Precedence is deliberate: values in `.env` **override** inherited shell variables. A stale key exported in `~/.bashrc` must not silently outrank the project's key. Shell values are used only for names `.env` does not define.

Fingerprinting

Never log a key. `fingerprint(secret)`:

- unset → `not set`;
- length <= 12 → all `*` plus `(<n> chars)`;
- otherwise `<first 5>...<last 4> (<n> chars)`.

Model normalisation

OpenRouter requires `vendor/model`. If the value already contains `/`, leave it.

Accepted aliases:

- key: `OPENROUTER_API_KEY` or `OPENROUTER_KEY`;
- model: `OPENROUTER_MODEL` or `MODEL`.

For bare names, use `OPENROUTER_PROVIDER_PREFIX` if set. Otherwise map prefixes:

PREFIX	VENDOR
`deepseek`	deepseek
`claude`	anthropic
`gpt`, `o1`, `o3`, `o4`, `o5`, `o8`, `chatgpt`	openai
`gemini`	google
`grok`	x-ai
`llama`	meta-llama
`qwen`	qwen
`mistral`, `mixtral`	mistralai

Return `{ model, normalised }`.

`loadConfig(overrides = {}, { envFile, startDir })`

Remember the inherited shell values before loading `.env`, then return:

```
{
  apiKey,
  apiKeySource,           // 'override' | '.env' | 'environment' | 'not set'
  apiKeyOverrodeShell,    // boolean: .env replaced a different inherited value
  model,
  modelSource,            // 'flag' | '.env' | 'environment' | 'not set'
  modelOverrodeShell,
  modelNormalised,
  workspaceRoot,
  commandPolicy,
  maxSteps,
  requestTimeoutMs,
  memoryFile,
  projectRoot,
  memoryMaxEntries,
  memoryPromptChars,
  envFile,
}
```

Defaults:

- model: `deepseek/deepseek-v4.1-flash-1017`
- `CMD_POLICY`: `ask` (`allow` | `ask` | `deny`)
- `MAX_STEPS`: 12
- `REQUEST_TIMEOUT_MS`: 90000
- `WORKSPACE_DIR`: `workspace` (resolved against project root)
- `MEMORY_FILE`: `src/memory.md` (relative resolves against project root; absolute is honoured)
- `MEMORY_MAX_ENTRIES`: 60
- `MEMORY_PROMPT_CHARS`: 1200; an explicit `0` disables injection but leaves `recall` working.

CLI overrides (`--model`, explicit `loadConfig` overrides) win over `.env`, which wins over the shell, which wins over defaults.

---

10. `ui.js` — all terminal styling

Export `ui` and `explainExit`. Use `chalk` only here.

Every helper returns a string and writes nothing. This keeps `runTask` logger-injectable. Chalk handles TTY/`NO_COLOR`/`FORCE_COLOR` itself.

Palette/API

```
ui.title(text)                // bold cyan
ui.field(label)               // green label, white ›
ui.kv(label, value)           // blue padded label + white value
ui.step(step, max, width=40)  // blue "── step X/Y ────"
ui.think(text)                // "think:" cyan + text
ui.tool(name, args)           // "tool →" white, name yellow, args white
ui.result(text)               // green/red result line, with details below
ui.forbidden(action, requested, reason)
ui.pass(text) / ui.fail(text)
ui.rule(label, width=44)      // bold magenta divider
ui.hookTitle(label)
ui.hookLine(icon, name, colour, when, live)
ui.hookField(label, value, colour)
ui.hookLive()
ui.hookNote(name, text)
ui.hookTrace(icon, name, colour, explanation, detail)
ui.hookFlow(steps)
ui.memory(text)
ui.exitNote(code, meaning)
ui.answerRule()               // bold green "── final answer ────"
ui.answer(text)
ui.warn / waiting / error / note / prompt / dim
```

Result rendering

`ui.result(text)` classifies failure when the first line starts with `ERROR`, `DENIED`, or `FORBIDDEN`, or contains a non-zero `exit code N`.

- Success: green `result ← <first line, 300 chars>`; if `exit code 0` includes a `stdout:` block, add a dim second line `↳ stdout: <first non-empty stdout line, 160 chars>`.
- Failure: red `result ← <first line>`; if it contains `exit code N` with N ≠ 0, add dim `↳ exit N: <meaning>`.

`explainExit(code)`

Return plain-English meanings for at least:

- 1: general failure; cause in stderr, fix it.
- 2: usage error / bad arguments or missing file.
- 3: Python argparse invalid arguments; bash bad builtin.
- 126: not executable / permission denied.
- 127: command not found / not on PATH.
- 128: invalid exit status outside 0–255.
- 129: SIGHUP.
- 130: SIGINT / Ctrl-C.
- 137: SIGKILL / OOM / timeout.
- 143: SIGTERM / killed politely.
- 9009: Windows cmd "not recognized as a program or batch file".
- 129–164: killed by signal `n - 128`.
- Else: non-zero exit; stdout/stderr say why.

`ui.forbidden` must be conspicuous:

```
⛔ FORBIDDEN SAVE ../escape.txt - path escapes the workspace: ../escape.txt
```

Bold red `FORBIDDEN`, bold white action in caps, red requested path, dim reason. It must be findable in a long transcript and greppable in a log.

Because chalk styles exact strings, model output containing colour-looking tags or Windows backslashes must print verbatim.

---

11. `heartbeat.js` — liveness

Export `heartbeat(log, everyMs = 5000)`.

- Record a start time.
- Every interval, print `ui.waiting(' … still waiting for the model (<seconds>s)')`.
- Return `{ stop }` which clears the interval.
- Call `timer.unref?.()` so the timer never holds the process open for an extra interval.
- The loop must stop the heartbeat on both the success and failure paths.

---

12. `demo.js` — offline scripted model

Export `DEMO_TASK`, `DEMO_SCRIPT`, `demoChat`, `sandboxScorecard`.

`demoChat()` returns an async `chatFn` with the real shape `{ choices: [{ finish_reason, message }] }`. It walks `DEMO_SCRIPT` once, then repeats the final answer so the loop can exit via the no-tool-calls path. Tool calls are `{ id, type: 'function', function: { name, arguments: JSON.stringify(args) } }`.

Demo script: 14 tool steps, then a final answer

The demo must exercise ordinary work **and** every important guard. Use this sequence:

1. **s1** `list_files` `{ path: '.' }` — look first.
2. **s2** `create_file` `{ path: 'demo/hello.js', content: "const agent = 'tiny agent';\\nconsole.log( Hello from a ${agent} );\\n" }`.
3. **s3** `run_command` `{ command: 'node demo/hello.js' }`.
4. **s4** `edit_file` `{ path: 'demo/hello.js', old_string: 'Hello from a ${agent}', new_string: 'Hello from a ${agent}, edited by a tool call' }`.
5. **s5** `run_command` `{ command: 'node demo/hello.js' }`.
6. **s6** `write_file` `{ path: '../escape.txt', content: 'this file must never be written' }` → expect FORBIDDEN.
7. **s7** `read_file` `{ path: '../package.json', limit: 5 }` → allowed; reads are open.
8. **s8** `write_file` `{ path: '~/Desktop/pwned.txt', content: 'the sandbox should have stopped this' }` → expect FORBIDDEN.
9. **s9** `run_command` `{ command: 'copy demo/hello.js %USERPROFILE%\\Desktop\\hello-copy.js' }` → expect FORBIDDEN for the shell write target.
10. **s10** `run_command` `{ command: 'git push --force origin main' }` → expect hard DENIED.
11. **s11** `run_command` `{ command: 'node -e "console.log(\'OPENROUTER_API_KEY=\' + (process.env.OPENROUTER_API_KEY ?? \'absent\'))"' }` → expect key absent from child environment.
12. **s12** `run_command` `{ command: 'node -e "process.exit(1)"' }` → expect exit code 1.
13. **s13** `remember` `{ text: 'The sandbox demo script lives at demo/hello.js and is safe to regenerate with create_file.', tag: 'fact' }`.
14. **s14** `recall` `{ query: 'demo' }`.

Then a final assistant message summarizing: created and ran `demo/hello.js`, edited and re-ran it with exit 0; the escape/home writes and shell copy were refused with FORBIDDEN before touching disk; the read one level up worked because reads are open; force push denied; child process could not see the API key; failing command returned exit 1 with an explanation; the scorecard is checkable.

Every assistant step should have a short natural-language `content` before its tool call, so the console shows the model "thinking" in prose.

The demo's `maxSteps` is `DEMO_SCRIPT.length + 1` (15), enough for all steps plus the final answer.

Scorecard

`DEMO_CHECKS` must verify at least:

CALL	CHECK
s6	`/^FORBIDDEN: SAVE \.\.\/escape\.txt/`
s7	`/^\.\.\/package\.json \(\d+ lines\)/`
s8	`/^FORBIDDEN: SAVE ~\/Desktop\/pwned\.txt/`
s9	`/^FORBIDDEN: COMMAND COPY /`
s10	`/^DENIED: blocked by policy/`
s11	`/OPENROUTER_API_KEY=absent/`, display `/OPENROUTER_API_KEY=\S+/`
s12	`/^exit code 1\b/`

`sandboxScorecard(messages)` maps `tool_call_id` → tool content, runs each regex, and returns `{ rows: [{ ok, label, seen }], passed, total }`. `seen` is the matched evidence or the first line, max 120 chars.

The scorecard is printed after a demo run. If any check fails, print the failing rows in red and set `process.exitCode = 1`.

13. `cli.js` — the program around the loop

This is the only file that knows about flags, the banner, prompts, sessions, `.env` presence, and exit codes.

`makeConfirm()`

Returns `async (command) => 'allow' | 'deny'`.

- If `process.stdin.isTTY` is false → `'deny'`. An unanswerable prompt must never become an approval.
- Otherwise use `readline` to ask:

```
run? <command> [y/N]
```

- Only `/^y(es)?$/i` on the trimmed answer is allow; everything else (including empty) is deny.

`askTask({ input, output })`

- Create `readline` with `terminal: Boolean(output.isTTY)`.
- `rl.on('SIGINT', () => rl.close())`.
- Prompt: `<green task> <white ››>`.
- Use `for await (const raw of rl)` rather than `rl.question()`, so multiple lines arriving in one chunk are queued, not dropped.
- Trim. Blank lines print `nothing entered - type a task, or press Ctrl-C to quit` and prompt again.
- Return `null` on closed input / Ctrl-C.
- Always close the interface in `finally`.

Argument parsing

Flags:

```
--demo
--yes | -y
--model <slug>
--max-steps <n>
--allow-commands
--hooks
--memory
--no-memory
--trace-hooks
--no-hooks
--help | -h
```

- All non-flags are joined with spaces into `task`, so quoting is optional.
- Unknown flags are not treated as errors; they become task text.
- `--max-steps` uses `Number(...)`.

Help/usage

Print a concise usage covering:

```
node src/cli.js                          interactive session
node src/cli.js "task"                   one shot
node src/cli.js --demo                   scripted run, no network
node src/cli.js "..." --yes              auto-approve commands
node src/cli.js "..." --allow-commands
node src/cli.js "..." --max-steps 6
node src/cli.js "..." --model anthropic/claude-sonnet-4.5
node src/cli.js --hooks
node src/cli.js "..." --no-hooks
node src/cli.js --memory
node src/cli.js "..." --no-memory
```

List environment variables:

```
OPENROUTER_API_KEY, OPENROUTER_MODEL, CMD_POLICY=allow|ask|deny, MAX_STEPS,
REQUEST_TIMEOUT_MS, MEMORY_FILE=src/memory.md, MEMORY_MAX_ENTRIES=60,
MEMORY_PROMPT_CHARS=1200
```

State that files are confined to `./workspace`, memory lives in `./src/memory.md`, and `npm test` is the offline suite. Explain session commands (`/new`, `/quit`) and exit codes 0/1/2/3.

Side doors

- `--hooks`: print the hook catalogue and a hint to attach handlers with `hooks.on(...)`; exit 0. Needs no config or key.
- `--memory`: load config and memory, then print:
- title;
- `file`, `exists`, `entries N (cap X, 300 chars each)`;
- `injected` — either disabled by `MEMORY_PROMPT_CHARS=0`, or `<n> newest, <digest.length> of <budget> chars`;
- the digest under `what the system prompt gets`;
- the whole raw file under `the whole file - <label>`;
- a note about `remember`/`recall`, hand editing, and `--no-memory`. Exit 0.

Banner

Print before step 1:

```
min-coding-js
workspace : <root>
sandbox   : save, edit, delete - only inside <root> (reads: anywhere)
model     : <model> (from <source>) | scripted (no network)
key       : <fingerprint> from <source> | - in demo
env file  : <path> | <path> (missing)
commands  : allow | ask | deny
timeout   : <seconds>s per model call
tools     : <comma-separated TOOL_NAMES>
memory    : <summary> | off (--no-memory)
hooks     : on: each of 9 prints as [HOOK:name] ... | 9 available, not printed (--no-hooks)
```

Warnings:

- live mode and model lacks `/` → `warning: model "<model>" is not an OpenRouter slug (needs vendor/model). Set OPENROUTER_MODEL in .env.`
- config normalised a bare name → a dim note with the final slug.

Wiring

1. Parse args; `--help` prints usage and returns.
2. `--hooks` prints catalogue and returns.
3. `loadConfig({ model: opts.model })`.
4. `new Workspace(config.workspaceRoot)`.
5. `new Sandbox({ workspace, log: console.log })` — one fence per run.
6. `new Memory({ root: config.projectRoot, file: config.memoryFile, maxEntries, promptChars })`.
7. `--memory` prints and returns.
8. Command policy: `opts.yes || opts.demo ? 'allow' : (opts.policy ?? config.commandPolicy)`.
9. Print banner.
10. `chatFn = opts.demo ? demoChat() : (args) => chat({ apiKey, model, timeoutMs, ...args })`.
11. Live mode with no key: print `OPENROUTER_API_KEY is not set. Copy .env.example to .env, or run --demo.`, set exit code 1, return.
12. Decide one-shot versus session:
    - task on command line, or demo → one task (`DEMO_TASK` for demo);
    - otherwise session.
13. Print `task : <task>` before a one-shot/demo.
14. Hooks: `opts.noHooks ? null : traceHooks({ log: console.log })`. When tracing:
    - print `<n> lifecycle hooks attached - each prints as it fires, e.g.`;
    - print one example `ui.hookTrace('🛠', 'PreToolUse', 'yellowBright', 'before every tool call', 'run_command {"command":"ls"}')`;
    - print `(--no-hooks hides these; --hooks explains them)`.
15. Build `base`:

```
{
  workspace, sandbox, commandPolicy,
  maxSteps: opts.maxSteps ?? (opts.demo ? DEMO_SCRIPT.length + 1 : config.maxSteps),
  chatFn,
  confirm: makeConfirm(),
  hooks,
  memory: opts.noMemory ? null : memory,
}
```

16. One-shot: `runTask({ ...base, task })`; set exit code; print sandbox scorecard for demo; return.
17. No task and stdin not a TTY: print `stopped: cannot ask for a task because stdin is not a terminal.`, a dim hint to pass a task argument, set exit code 2, return.
18. Session: `runSession({ base, opts })`.

Sessions

Export `MAX_HISTORY = 24`, `interpretCommand`, `trimHistory`, `runSession`.

Commands

- `/quit` or `/exit` → quit.
- `/new` → new conversation (clear history; workspace and memory untouched).
- Anything else → task.

`trimHistory(messages, max = MAX_HISTORY)`

- Drop system messages.
- Keep at most the last `max` messages.
- Move `start` forward until it lands on a `user` message. Providers reject a history that starts with a `tool` result or ends with an unanswered assistant `tool_calls`.
- If no safe boundary exists, return `[]`.

`runSession`

- Print `interactive session - type a task below (/new clears, /quit exits)`.
- Loop:
- `askTask()`; `null` → break.
- Handle command; `/new` clears history and says the workspace and memory are untouched.
- Print `task : <line>`.
- `runTask({ ...base, task: line, history })`.
- `history = trimHistory(result.messages)`.
- For demo, print sandbox scorecard.
- Print a summary:
  - divider `task finished`;
  - green pass `<answer in N step(s)>` or red fail `<stopped (<reason>)>`;
  - note: success → `<n> message(s) carried into the next task`; max steps → `raise it with --max-steps N, or /new for a fresh conversation`; otherwise → `check the red line above - the session is still alive, so you can try again`;
  - hint: `type another task, /new to clear the conversation, /quit to exit`.
- On exit: `session over - N task(s) run. The workspace and src/memory.md keep everything.`
- Every task gets a fresh step budget because every task is a fresh `runTask` call. History resets the step counter by construction.

Exit codes

CODE	MEANING
0	Finished / model gave a final answer / session ended normally
1	Model request failed (network/timeout/key/model) or a fatal error; nothing retried
2	No task and stdin is not a terminal, so the prompt cannot be shown
3	Step limit reached before the model answered

`taskExitCode(result)`:

- `!ok && reason === 'request_failed'` → 1;
- `!ok && reason === 'max_steps'` → 3;
- otherwise 0.

`main()` catches fatal errors, prints `fatal: <stack>`, sets exit code 1. If the final code is non-zero, print a blank line and `ui.exitNote(code, meaning)` with plain-English meanings:

- 1: run did not finish; a model request failed or a fatal error was caught; the red line above says which, nothing retried.
- 2: no task given; nothing ran.
- 3: step limit reached; raise `--max-steps` / `MAX_STEPS` or give a smaller task.

Only call `main()` when `cli.js` is the entry file (`import.meta.url === pathToFileURL(process.argv[1]).href`).

14. Testing requirements

Use `node:test` and `node:assert/strict`. Tests must be offline: no API key, no network, no live model. Use temporary directories, fake `chatFn`s, injected clocks, and fake loggers.

Provide suites covering at least:

`workspace.test.js`

- Resolves legitimate relative paths; `rel` is correct.
- Refuses every escape shape: `../`, `..\\`, absolute, drive-relative, UNC, `~`, `%USERPROFILE%`, `${HOME}`, home/system first segments.
- Allows reads outside the workspace.
- A symlinked folder cannot smuggle a write outside the root.

`sandbox.test.js`

- Default sandbox is `workspace/` next to the code.
- `save`, `edit`, `remove` work inside.
- A refused write prints exactly one FORBIDDEN line, increments refusals, touches nothing.
- FORBIDDEN is bold red capitals when colour is on.
- Every escape shape is refused for all three actions.
- Deleting the root is refused; deleting outside leaves the file intact.
- Unknown action throws a programming `TypeError`, not a refusal.
- The tool layer turns a refusal into a `FORBIDDEN:` observation.

`tools.test.js`

- `create_file` writes and refuses to clobber.
- `write_file` overwrites.
- `edit_file` fails on missing and ambiguous snippets.
- `edit_file` treats `$1` / `$&` literally.
- `read_file` numbers lines and reports total.
- `list_files` shows folders/files with counts.
- `run_command` returns exit code, stdout, stderr.
- `run_command` obeys deny/ask-denied/hard-blocked policies.
- File tools cannot be talked out of the workspace.
- Reads outside succeed.
- Child gets the allow-list, not the harness environment.
- `safeEnv` keeps only allow-listed names.
- Bad arguments come back as text, not crashes.

`shell-sandbox.test.js`

- `shellWriteTargets` finds redirect, copy/move destination, delete operands, downloader output, PowerShell targets.
- A command writing inside the sandbox is allowed.
- A shell copy to the Desktop is refused before it runs.
- A redirect outside is refused.
- Quoted arguments with spaces stay one token.

`agent.test.js`

- The loop runs tools then stops on a plain answer.
- A failing tool result is fed back so the model can correct itself.
- The loop is capped at `maxSteps`.
- A truncated reply is not dispatched; the model is asked to continue smaller.
- A slow model call prints heartbeat lines.
- The scripted demo tries every guard, and every guard holds.

`hooks.test.js`

- Every hook has icon, colour, when, gets, may, why.
- Catalogue prints every hook with icon/colour/description.
- Catalogue marks attached hooks.
- Unknown hook name throws on `on` and on `run`.
- Handlers run in order; first block wins.
- A throwing hook is reported and ignored.
- Notes print; `once` handlers detach.
- `off` by id works.
- Trace prints `[HOOK:Name]` and description; `explain: false` prints tag + detail only.
- `hookDetail` gives every event something useful.
- `PreToolUse` can block and the model sees a DENIED observation.
- `PostToolUse` can rewrite the observation.
- `UserPromptSubmit` can rewrite the task.
- `SessionStart` can refuse to start without any model request.
- A live run fires the whole lifecycle; `SessionEnd` is always last.
- Step limit fires `StepLimit`, not `Stop`, before `SessionEnd`.
- No hooks behaves exactly as before.

`memory.test.js`

- Write and read back.
- Survives a new `Memory` instance / process.
- Duplicate refreshes date instead of appending.
- `maxEntries` drops oldest.
- Long memory truncated to per-entry cap.
- Empty/whitespace note refused.
- Digest bounded, newest first, states hidden count.
- Empty digest is empty.
- Search is AND, newest first, searches whole file.
- `forget` removes matches.
- Prose and hand-written bullets parse.
- Hand edits picked up immediately (no cache).
- Label relative to root; absolute file honoured.
- `remember` and `recall` work through the tools.
- No memory store returns an error instead of throwing.
- `formatEntry` round-trips through `parseMemory`.

`prompt.test.js`

- Rules name every tool.
- No memory → rules only.
- Memory appended; log receives exactly one memory line.
- Empty/disabled store adds and logs nothing.

`llm.test.js`

- A stalled request times out instead of hanging.
- Provider error includes status and body.
- `sanitizeAssistant` drops reasoning and rebuilds the message.

`config.test.js`

- `.env` beats a stale shell key.
- Shell used for names `.env` does not define.
- Missing `.env` is not an error.
- Quotes, `export`, and blank values handled.
- `.env` is found in the project root when the package has none.
- Nearest `.env` wins.
- Fingerprint never exposes the whole key.
- Bare model names normalised to vendor/model.
- Config normalises `.env` model.
- `.env MODEL=` honoured and beats inherited `OPENROUTER_MODEL`.
- Key from `.env` used and override reported.
- Identical values not reported as override.

`session.test.js`

- `/quit`, `/exit`, `/new` commands; anything else a task.
- History trimmed to cap and always starts on a user message.
- Follow-up task sees previous conversation but a fresh system prompt.
- Every task starts its own step budget.
- A hook rewriting the task rewrites the new task, not history.

`ui.test.js`

- Styles become ANSI but read back as original text.
- Text that looks like a colour tag prints verbatim.
- Backslashes in paths/JSON untouched.
- Failed results red; clean results not.
- Exit code 1 gets a description.
- Successful command shows first stdout line.
- Every helper returns a string.

`input.test.js`

- Input field returns trimmed task.
- Blank line re-asks.
- Closed input returns `null`.

---

15. README and teaching material

Write a README that a meetup attendee can follow:

- What the project is, in one paragraph.
- Quick start: `npm install`, copy `.env.example` to `.env`, set `OPENROUTER_API_KEY`, `npm start`, `npm run demo`.
- The loop diagram and the seven beats.
- File-by-file tour, with `agent.js` first.
- The eight tools and what each is for.
- The read/write asymmetry and the shell limitation.
- The nine lifecycle hooks, with a table.
- Long-term memory: file format, eager digest vs lazy recall, `--memory`.
- Sessions and history trimming.
- Configuration table with defaults and precedence.
- Exit codes.
- Safety model: workspace fence, hard denies, environment allow-list, command policy, and an explicit "this is not a container" statement.
- Examples, including a Desktop write that should end in FORBIDDEN, and a prompt that should end with a `remember` call.

`demo_prompt.md` should contain copy-pasteable prompts, including:

```
create hello.js that prints today's date and time, run it and save to current folder with a random number between 100 and 999 as helllоNNN.js where NNN is that random number printing out in console location of the file and use that for the saved output as hello-output-NNN.txt
```

```
save a file test.txt that contains 'HELLO WORLD' and save to the desktop
```

The second must end in `FORBIDDEN: SAVE ... - the Desktop is outside the workspace`.

Also include:

```
npm run demo
node src/cli.js --memory
node src/cli.js --demo
```

and:

```
work out how this package is tested, run the tests, and remember the command and the Node version it needs so you do not have to rediscover them next time
```

---

16. Acceptance criteria

The build is complete when all of the following are true:

1. `npm install` succeeds with only `chalk` as a runtime dependency.
2. `npm test` runs fully offline and all suites pass.
3. `node src/cli.js --help` prints usage without needing a key.
4. `node src/cli.js --hooks` prints the full nine-hook catalogue without needing a key.
5. `node src/cli.js --memory` prints the store and the exact digest that would be injected.
6. `node src/cli.js --demo` completes the 14-step script: - creates/edits/runs `workspace/demo/hello.js`; - refuses the `../escape.txt` write; - allows the `../package.json` read; - refuses the `~/Desktop/pwned.txt` write; - refuses the shell copy to the Desktop; - denies the force push; - shows `OPENROUTER_API_KEY=absent` from the child; - reports exit code 1 with an explanation; - remembers and recalls a fact; - prints a scorecard with **7/7 checks held**; - exits 0.
7. Live mode with a valid key runs a task through OpenRouter and prints step headers, tool calls, tool results, and a final answer.
8. Live mode without a key prints the missing-key error and exits 1.
9. A live/session run without a task and without a TTY exits 2.
10. A task that never yields a final answer stops at the step limit and exits 3.
11. A write outside `workspace/` prints one `⛔ FORBIDDEN ...` line and returns a `FORBIDDEN:` observation to the model without touching disk.
12. A read outside `workspace/` succeeds.
13. `CMD_POLICY=deny` refuses all commands; hard-denied commands are refused even with `--yes`.
14. The child shell cannot read `OPENROUTER_API_KEY`.
15. Memory survives process restarts; duplicate notes refresh instead of duplicating; the file is bounded; a hand edit is picked up immediately.
16. Every normal run prints the `[HOOK:Name]` trace by default; `--no-hooks` silences it.
17. `/new` clears conversation history but leaves workspace and memory untouched; `/quit` and `/exit` end the session.
18. A follow-up task sees the previous conversation but a freshly rebuilt system prompt containing the latest memory.
19. `agent.js` executed directly prints the "this is the loop, not the program" hint and does not run the CLI.
20. The code contains no framework, no agent SDK, no database, and no hidden global state.

---

17. Suggested implementation order

1. `workspace.js` and its tests — path policy first.
2. `ui.js` and `ui` tests — establish the visual language.
3. `sandbox.js` + shell target scanner and tests.
4. `memory.js` + tests.
5. `prompt.js` + tests.
6. `tools.js` + tests.
7. `llm.js` + tests.
8. `hooks.js` + tests.
9. `agent.js` + fake-model tests.
10. `demo.js` and scorecard.
11. `config.js` + tests.
12. `cli.js` + session/input tests.
13. README, demo prompts, final end-to-end demo run.

---

18. Tone and design philosophy (important)

- **Small enough to explain.** One loop, eight tools, nine hooks. Resist adding features.
- **One obvious place for each decision.** Config in `config.js`, path policy in `workspace.js`, mutation gate in `sandbox.js`, prompt in `prompt.js`, presentation in `ui.js`, process concerns in `cli.js`.
- **Observability over magic.** Every step, tool call, result, hook firing, and refusal is visible in the terminal.
- **Bounded autonomy.** Step limits, output caps, timeouts, per-entry memory caps, prompt-memory budgets, and a deny-list all exist because "autonomous" without bounds is a bug.
- **Honest safety.** The workspace fence and shell write scanner are useful but not a container. Say so in code comments and docs.
- **Human in the loop.** Default command policy is `ask`; no TTY means deny; hard denies cannot be overridden.
- **Testable by construction.** Inject the model, clock, logger, confirm function, hooks, memory, and workspace. If a feature cannot be tested offline, the seam is wrong.
- **Teachable error strings.** A refusal, a failed exit code, and a missing key must explain what happened and what to do next.
- **No fake results.** The system prompt forbids invented tool results, and every tool observation is a real string produced by the harness.
- **Memory is a file you can read and diff.** No vector store, no embeddings, no hidden index. A student can open `src/memory.md` and see exactly what the agent knows.